

PDF与Word文档解析技术调研： PyMuPDF & python-docx

目录

- PDF与Word文档解析技术调研： PyMuPDF & python-docx

- 1. 概述

- 1.1 调研背景

- 1.2 库简介

- 2. 安装

- 2.1 PyMuPDF

- 2.2 python-docx

- 2.3 完整依赖安装（推荐）

- 3. PyMuPDF 详细调研

- 3.1 核心概念

- 3.2 基础用法

- 3.3 结构化文本提取

- 3.4 按章节切分 PDF（基于标题识别）

- 3.5 利用 PDF 目录（TOC）切分

- 3.6 图片提取

- 4. python-docx 详细调研

- 4.1 核心概念

- 4.2 基础用法

- 4.3 识别段落样式与标题

- 4.4 按章节切分 Word 文档

- 4.5 提取完整文档结构

- 4.6 创建结构化 Word 文档
- 5. 综合对比
- 6. 章节切分最佳实践总结
 - PDF
 - Word
- 7. 快速上手模板
- 8. 参考资料

PDF与Word文档解析技术调研：PyMuPDF & python-docx

1. 概述

1.1 调研背景

在文档智能处理场景中，PDF 和 Word 文档是最常见的两种格式。本文调研 PyMuPDF（处理PDF）和 python-docx（处理Word）两个主流 Python 库，涵盖安装配置、核心API、章节切分等实战场景。

1.2 库简介

1	特性	PyMuPDF	python-docx
2	目标格式	PDF、XPS、EPUB 等	.docx（Word 2007+）
3	底层依赖	MuPDF C库	lxml
4	许可证	AGPL-3.0 / 商业授权	MIT

5	PyPI 包名	pymupdf	python-docx
6	活跃度	高	高

2. 安装

2.1 PyMuPDF

```
</> Bash |  
1 pip install pymupdf
```

注意：包名是 `pymupdf`，但 import 时使用 `fitz`（历史遗留）。Python 3.8+ 支持；无需单独安装 MuPDF，wheel 包已内置。

2.2 python-docx

```
</> Bash |  
1 pip install python-docx
```

注意：包名是 `python-docx`，import 时使用 `docx`。依赖 lxml，通常会自动安装。

2.3 完整依赖安装（推荐）

```
</> Bash |  
1 pip install pymupdf python-docx
```

3. PyMuPDF 详细调研

3.1 核心概念

- **Document**: 文档对象，对应一个 PDF 文件
- **Page**: 页面对象，包含文本、图像、注释等
- **Block**: 文本块，是页面内容的基本组织单元
- **Span**: 行内文本片段，携带字体、大小、颜色等样式信息

3.2 基础用法

```
</> Python |  
  
1 import fitz # PyMuPDF  
2  
3 # 打开文档  
4 doc = fitz.open("example.pdf")  
5 print(f"总页数: {doc.page_count}")  
6 print(f"元数据: {doc.metadata}")  
7  
8 # 遍历页面提取文本  
9 for page_num, page in enumerate(doc):  
10     text = page.get_text()  
11     print(f"第 {page_num + 1} 页:\n{text[:200]}")  
12  
13 doc.close()
```

3.3 结构化文本提取

```
1 import fitz
2
3 def extract_structured_text(pdf_path: str) -> list[dict]:
4     """提取带样式信息的结构化文本"""
5     doc = fitz.open(pdf_path)
6     pages_data = []
7
8     for page_num, page in enumerate(doc):
9         # dict 模式返回完整结构: blocks > lines > spans
10        page_dict = page.get_text("dict")
11        blocks = []
12        for block in page_dict["blocks"]:
13            if block["type"] != 0: # 0=文字块, 1=图片块
14                continue
15            for line in block["lines"]:
16                for span in line["spans"]:
17                    blocks.append({
18                        "text": span["text"],
19                        "font": span["font"],
20                        "size": span["size"],
21                        "bold": bool(span["flags"] & 2**4),
22                        "italic": bool(span["flags"] & 2**1),
23                        "color": span["color"],
24                        "bbox": span["bbox"],
25                    })
26        pages_data.append({"page": page_num + 1, "spans": blocks})
27
28    doc.close()
```

3.4 按章节切分 PDF（基于标题识别）

</>

Python

```
1 import fitz
2 from dataclasses import dataclass, field
3
4 @dataclass
5 class Section:
6     title: str
7     level: int
8     page_start: int
9     content: list[str] = field(default_factory=list)
10
11
12 def detect_heading_level(span: dict, avg_body_size: float) -> int:
13     """根据字体大小和加粗判断标题级别, 返回 0 表示正文"""
14     size = span["size"]
15     bold = bool(span["flags"] & 2**4)
16
17     if size >= avg_body_size * 1.8 or (bold and size >= avg_body_size * 1.5):
18         return 1
19     if size >= avg_body_size * 1.4 or (bold and size >= avg_body_size * 1.2):
20         return 2
21     if bold and size >= avg_body_size * 1.05:
22         return 3
23     return 0
```

```
24
25
26 def split_pdf_by_sections(pdf_path: str) -> list[Section]:
27     """将 PDF 按章节切分"""
28     doc = fitz.open(pdf_path)
29
30     # 第一遍：统计正文平均字号（取中位数）
31     sizes = []
32     for page in doc:
33         for block in page.get_text("dict")["blocks"]:
34             if block["type"] != 0:
35                 continue
36             for line in block["lines"]:
37                 for span in line["spans"]:
38                     if span["text"].strip():
39                         sizes.append(span["size"])
40
41     avg_size = sorted(sizes)[len(sizes) // 2] if sizes else 12.0
42
43     # 第二遍：按页提取并识别章节
44     sections: list[Section] = []
45     current_section: Section | None = None
46
47     for page_num, page in enumerate(doc):
48         for block in page.get_text("dict")["blocks"]:
49             if block["type"] != 0:
50                 continue
51             for line in block["lines"]:
52                 line_text = "".join(s["text"] for s in line["spans"]).strip()
```

```
53         if not line_text:
54             continue
55
56         max_span = max(line["spans"], key=lambda s: s["size"])
57         level = detect_heading_level(max_span, avg_size)
58
59         if level > 0:
60             if current_section:
61                 sections.append(current_section)
62                 current_section = Section(
63                     title=line_text,
64                     level=level,
65                     page_start=page_num + 1
66                 )
67             elif current_section is not None:
68                 current_section.content.append(line_text)
69
70     if current_section:
71         sections.append(current_section)
72
73     doc.close()
74     return sections
75
76
77 # 使用示例
78 sections = split_pdf_by_sections("document.pdf")
79 for sec in sections:
80     print(f"{'#' * sec.level} [{sec.page_start}页] {sec.title}")
81     print("\n".join(sec.content[:3]), "...\\n")
```


3.5 利用 PDF 目录 (TOC) 切分

</>

Python

```
1 import fitz
2
3 def split_by_toc(pdf_path: str) -> list[dict]:
4     """优先使用 PDF 内嵌目录切分章节 (比字体启发式更准确) """
5     doc = fitz.open(pdf_path)
6     toc = doc.get_toc() # [[level, title, page], ...]
7
8     if not toc:
9         print("该 PDF 无内嵌目录, 请使用字体启发式方法")
10        return []
11
12    sections = []
13    for i, (level, title, page_start) in enumerate(toc):
14        page_end = toc[i + 1][2] - 1 if i + 1 < len(toc) else doc.page_count
15
16        content_parts = []
17        for pno in range(page_start - 1, min(page_end, doc.page_count)):
18            content_parts.append(doc[pno].get_text())
19
20        sections.append({
21            "level": level,
22            "title": title,
23            "page_start": page_start,
24            "page_end": page_end,
25            "content": "\n".join(content_parts),
26        })
```

```
27
28     doc.close()
29     return sections
```

3.6 图片提取

</>

Python

```
1 import fitz, os
2
3 def extract_images(pdf_path: str, output_dir: str = ".") -> list[str]:
4     """提取 PDF 中所有图片"""
5     doc = fitz.open(pdf_path)
6     saved = []
7
8     for page_num, page in enumerate(doc):
9         for img_index, img in enumerate(page.get_images(full=True)):
10             xref = img[0]
11             base_image = doc.extract_image(xref)
12             ext = base_image["ext"]
13             path = os.path.join(output_dir, f"p{page_num+1}_img{img_index+1}.{ext}")
14             with open(path, "wb") as f:
15                 f.write(base_image["image"])
16             saved.append(path)
17
18     doc.close()
19     return saved
```

4. python-docx 详细调研

4.1 核心概念

- **Document**: 文档对象
- **Paragraph**: 段落, 是 Word 文档的基本内容单元
- **Run**: 段落内的连续格式文本片段
- **Style**: 样式, Heading 1/2/3 等用于识别标题层级
- **Table**: 表格对象

4.2 基础用法

```
</> Python |  
  
1 from docx import Document  
2  
3 doc = Document("example.docx")  
4  
5 # 遍历段落  
6 for para in doc.paragraphs:  
7     print(f"样式: {para.style.name:20} | 内容: {para.text[:60]}")
```

4.3 识别段落样式与标题

```
</> Python |  
  
1 def classify_paragraph(para) -> tuple[int, str]:  
2     """  
3     返回 (heading_level, text)
```

```

4 heading_level=0 表示正文
5 """
6 style_name = para.style.name # e.g. "Heading 1", "Normal"
7
8 if style_name.startswith("Heading"):
9     try:
10         level = int(style_name.split()[-1])
11     except ValueError:
12         level = 1
13     return level, para.text
14
15 # 兜底：检测加粗且字号较大的段落
16 if para.runs:
17     run = para.runs[0]
18     if run.bold and run.font.size and run.font.size.pt >= 14:
19         return 1, para.text
20
21 return 0, para.text

```

4.4 按章节切分 Word 文档

</>

Python

```

1 from docx import Document
2 from docx.table import Table as DocxTable
3 from dataclasses import dataclass, field
4
5 @dataclass
6 class DocSection:

```

```
7 title: str
8 level: int
9 paragraphs: list[str] = field(default_factory=list)
10 tables: list[list[list[str]]] = field(default_factory=list)
11
12
13 def split_docx_by_sections(docx_path: str) -> list[DocSection]:
14     """按标题层级切分 Word 文档（保持段落与表格的相对顺序）"""
15     doc = Document(docx_path)
16     sections: list[DocSection] = []
17     current: DocSection | None = None
18
19     # 构建 element -> paragraph 对象映射
20     para_map = {p._element: p for p in doc.paragraphs}
21
22     for element in doc.element.body:
23         tag = element.tag.split("}")[1] # 'p' or 'tbl'
24
25         if tag == "p":
26             para_obj = para_map.get(element)
27             if para_obj is None:
28                 continue
29             level, text = classify_paragraph(para_obj)
30             if not text.strip():
31                 continue
32
33             if level > 0:
34                 if current:
35                     sections.append(current)
```

```

36         current = DocSection(title=text, level=level)
37     elif current is not None:
38         current.paragraphs.append(text)
39
40     elif tag == "tbl" and current is not None:
41         tbl = DocxTable(element, doc)
42         table_data = [
43             [cell.text for cell in row.cells]
44             for row in tbl.rows
45         ]
46         current.tables.append(table_data)
47
48     if current:
49         sections.append(current)
50
51     return sections
52
53
54 # 使用示例
55 sections = split_docx_by_sections("document.docx")
56 for sec in sections:
57     print(f"{'#' * sec.level} {sec.title}")
58     for p in sec.paragraphs[:2]:
59         print(f"    {p[:80]}")
60     if sec.tables:
61         print(f"    [包含 {len(sec.tables)} 个表格]")

```

4.5 提取完整文档结构

```
1 from docx import Document
2
3 def extract_docx_structure(docx_path: str) -> dict:
4     """提取 Word 文档的完整结构（段落+表格+图片统计）"""
5     doc = Document(docx_path)
6     result = {"paragraphs": [], "tables": [], "image_count": 0}
7
8     for para in doc.paragraphs:
9         if para.text.strip():
10             result["paragraphs"].append({
11                 "style": para.style.name,
12                 "text": para.text,
13                 "runs": [
14                     {
15                         "text": r.text,
16                         "bold": r.bold,
17                         "italic": r.italic,
18                         "font_size": r.font.size.pt if r.font.size else None,
19                     }
20                     for r in para.runs if r.text
21                 ],
22             })
23
24     for table in doc.tables:
25         result["tables"].append(
26             [[cell.text for cell in row.cells] for row in table.rows]
27         )
28
```

```
29     result["image_count"] = len(doc.inline_shapes)
30     return result
```

4.6 创建结构化 Word 文档

</>

Python

```
1 from docx import Document
2
3 def create_structured_doc(output_path: str, sections: list[dict]):
4     """根据章节数据创建结构化 Word 文档"""
5     doc = Document()
6     for sec in sections:
7         doc.add_heading(sec["title"], level=sec.get("level", 1))
8         for para_text in sec.get("paragraphs", []):
9             doc.add_paragraph(para_text)
10    doc.save(output_path)
```

5. 综合对比

1	维度	PyMuPDF	python-docx
2	章节识别方式	字体大小/加粗启发式 or TOC	Heading 样式（精准）
3	表格支持	基础提取	完整读写
4	图片提取	支持（含位图/矢量）	仅内嵌图片
5	文本精度	依赖 PDF 质量	高（结构化存储）

6	修改文档	有限支持	完整支持
7	扫描件处理	需配合 OCR	不适用
8	性能	高 (C底层)	中等

6. 章节切分最佳实践总结

PDF

1. 优先使用 TOC: `doc.get_toc()` 获取内嵌目录, 准确率接近 100%
2. 无 TOC 时用字体启发式: 以页面所有字号的中位数为基准, 大于 1.4x 认定为标题
3. 多级标题阈值可配置化, 不同文档字号规范不同

Word

1. 优先依赖 Heading 样式: `para.style.name.startswith("Heading")` 最可靠
2. 兜底用字体特征: 无样式时检测加粗+大字号
3. 遍历 `doc.element.body` 而非 `doc.paragraphs`, 可保持段落与表格的相对顺序

7. 快速上手模板

</>

Python

```
1 # requirements.txt
2 # pymupdf>=1.24.0
3 # python-docx>=1.1.0
4
```

```
5 from pathlib import Path
6 import fitz
7 from docx import Document
8
9 def parse_document(file_path: str) -> list[dict]:
10     """统一入口：根据后缀自动选择解析器"""
11     suffix = Path(file_path).suffix.lower()
12     if suffix == ".pdf":
13         return parse_pdf(file_path)
14     elif suffix in (".docx", ".doc"):
15         return parse_docx(file_path)
16     else:
17         raise ValueError(f"不支持的文件类型：{suffix}")
18
19
20 def parse_pdf(path: str) -> list[dict]:
21     doc = fitz.open(path)
22     toc = doc.get_toc()
23     if toc:
24         result = []
25         for i, (level, title, page) in enumerate(toc):
26             end = toc[i+1][2]-1 if i+1 < len(toc) else doc.page_count
27             content = "\n".join(doc[p].get_text() for p in range(page-1, min(end, doc.page_count)))
28             result.append({"level": level, "title": title, "content": content})
29         doc.close()
30         return result
31     doc.close()
32     return [{"level": s.level, "title": s.title, "content": "\n".join(s.content)}
33             for s in split_pdf_by_sections(path)]
```

```
34
35
36 def parse_docx(path: str) -> list[dict]:
37     return [{"level": s.level, "title": s.title, "content": "\n".join(s.paragraphs)}
38             for s in split_docx_by_sections(path)]
```

8. 参考资料

- PyMuPDF 官方文档: <https://pymupdf.readthedocs.io/>
- python-docx 官方文档: <https://python-docx.readthedocs.io/>
- PyMuPDF GitHub: <https://github.com/pymupdf/PyMuPDF>
- python-docx GitHub: <https://github.com/python-openxml/python-docx>